

```

-- =====
-- COMIC BOOKS DATABASE | Stored Procedures (final)
-- Author: Michael Cummins
-- Run after all tables are created and data is loaded.
--
-- FULL REQUIREMENT COVERAGE:
--
-- SELECT      #1-13    DELETE      #8
-- UPDATE      #7, #10  INSERT      #6
-- LIMIT       #2       WHERE LIKE   #3, #4, #11
-- '_' wildcard #3, #11  '%' wildcard #3, #4, #11
-- COUNT()     #5, #9, #10, View1, View2
-- SUM()       #5, #9   AVG()        #5
-- MAX()       #5      MIN()        #5
-- GROUP BY    #5, #9, View1, View2, #13
-- ORDER BY    #1-13   DESC         #2, #9, #12
-- REPLACE     #3, #11  REVERSE    #3, #11
-- LENGTH()    #3, #11  LENGTH(TRIM()) #3, #11
-- CONCAT      #2, #8   CONCAT_WS   #2
-- AS          #1-13   SUBSTR      #3, #11
-- LEFT()      #3, #11  RIGHT()     #3, #11
-- POSITION()   #3, #11  CROSS JOIN  #13
-- INNER JOIN  #1, #2, #4, #9, #12
-- LEFT JOIN   #4, #5, View1, View2
-- RIGHT JOIN  View2    AND / OR    #2, #3, #7, #11
-- VIEW        View1, View2
-- HAVING      #9, #13  ROLLUP      #5
-- COALESCE    #4, #5, View2
-- PROCEDURE   #6, #7, #8, #10
-- FUNCTIONS   #1, #2, #3, #4, #5, #9, #11, #12, #13
-- OVER        #12     ROW_NUMBER() #12
-- RANK()      #12     DENSE_RANK() #12
-- LAG()       #12     LEAD()      #12
-- WITH AS     #12
--
-- EXTRA FEATURES (beyond requirements):
--
-- LOWER()     #1, #2, #3, #4, #7, #8, #11
-- DISTINCT    #5, #13
-- ROUND()     #5
-- RAISE EXCEPTION #3, #6, #7, #8, #10
-- RAISE NOTICE  #6, #7, #8, #10
-- GET DIAGNOSTICS #8
-- PARTITION BY #12

```

```

-- DECLARE      #6, #7, #8, #10
-- =====

-- =====
-- PROCEDURE 1 — get_issues_by_publisher
--
-- REQUIREMENTS MET:
--   SELECT, INNER JOIN (x2), WHERE, AS, ORDER BY
--
-- EXTRA FEATURES:
--   LOWER() — used for case-insensitive publisher name matching
--
-- Basic JOIN across 3 tables (issue, series, publisher).
-- Returns all issues for a given publisher name.
-- =====

CREATE OR REPLACE FUNCTION get_issues_by_publisher(p_publisher_name VARCHAR)
RETURNS TABLE (
    publisher  VARCHAR,
    series     VARCHAR,
    issue_number VARCHAR,
    issue_title VARCHAR,
    publish_date DATE,
    cover_price NUMERIC
)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
        SELECT
            p.name      AS publisher,
            s.title     AS series,
            i.issue_number AS issue_number,
            i.title     AS issue_title,
            i.publish_date,
            i.cover_price
        FROM issue i
        INNER JOIN series s ON i.series_id = s.series_id
        INNER JOIN publisher p ON s.publisher_id = p.publisher_id
        WHERE LOWER(p.name) = LOWER(p_publisher_name)
        ORDER BY s.title, i.publish_date;
END;
$$;

```

```

-- USAGE:
-- SELECT * FROM get_issues_by_publisher('Marvel Comics');
-- SELECT * FROM get_issues_by_publisher('DC Comics');

-- =====
-- PROCEDURE 2 — get_creator_credits
--
-- REQUIREMENTS MET:
-- SELECT, INNER JOIN (x3), WHERE, AND, LIMIT, ORDER BY DESC,
-- CONCAT, CONCAT_WS, AS
--
-- EXTRA FEATURES:
-- LOWER() — used for case-insensitive name matching
--
-- Returns every issue a creator worked on and their role.
-- Demonstrates both CONCAT and CONCAT_WS for string building
-- and limits to the 100 most recent credits.
-- =====

CREATE OR REPLACE FUNCTION get_creator_credits(
  p_first_name VARCHAR,
  p_last_name  VARCHAR
)
RETURNS TABLE (
  creator    TEXT,
  creator_ws TEXT,
  role       VARCHAR,
  series     VARCHAR,
  issue_number VARCHAR,
  issue_title VARCHAR,
  publish_date DATE
)
LANGUAGE plpgsql AS $$
BEGIN
  RETURN QUERY
  SELECT
    CONCAT(c.first_name, ' ', c.last_name) AS creator,
    CONCAT_WS(' ', c.first_name, c.last_name) AS creator_ws,
    ic.role,
    s.title AS series,
    i.issue_number,
    i.title AS issue_title,
    i.publish_date

```

```

FROM issue_creator ic
INNER JOIN creator c ON ic.creator_id = c.creator_id
INNER JOIN issue i ON ic.issue_id = i.issue_id
INNER JOIN series s ON i.series_id = s.series_id
WHERE LOWER(c.first_name) = LOWER(p_first_name)
      AND LOWER(c.last_name) = LOWER(p_last_name)
ORDER BY i.publish_date DESC
LIMIT 100;
END;
$$;

-- USAGE:
-- SELECT * FROM get_creator_credits('Stan', 'Lee');
-- SELECT * FROM get_creator_credits('Alan', 'Moore');

-- =====
-- PROCEDURE 3 — search_characters_by_name
--
-- REQUIREMENTS MET:
-- SELECT, WHERE, LIKE '%' wildcard, LIKE '_' wildcard, OR,
-- AS, ORDER BY, REPLACE, REVERSE, LENGTH, LENGTH(TRIM()),
-- SUBSTR, LEFT, RIGHT, POSITION
--
-- EXTRA FEATURES:
-- LOWER() — used inside POSITION() for case-insensitive search
--
-- Searches characters by a partial name pattern and returns
-- a rich set of string-function results on the name column.
-- Pass a search term and it will match anywhere in the name.
-- =====

CREATE OR REPLACE FUNCTION search_characters_by_name(p_search VARCHAR)
RETURNS TABLE (
    character_name VARCHAR,
    alignment      VARCHAR,
    name_replaced  TEXT,
    name_reversed  TEXT,
    name_length    INT,
    name_trimmed_len INT,
    name_substr    TEXT,
    name_left4     TEXT,
    name_right4    TEXT,
    pos_of_a       INT

```

```

)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
        SELECT
            ch.name                                AS character_name,
            ch.alignment,
            REPLACE(ch.name::TEXT, 'man', 'person') AS name_replaced,
            REVERSE(ch.name::TEXT)                 AS name_reversed,
            LENGTH(ch.name::TEXT)                   AS name_length,
            LENGTH(TRIM(ch.name::TEXT))             AS name_trimmed_len,
            SUBSTR(ch.name::TEXT, 1, 5)             AS name_substr,
            LEFT(ch.name::TEXT, 4)                  AS name_left4,
            RIGHT(ch.name::TEXT, 4)                 AS name_right4,
            POSITION('a' IN LOWER(ch.name::TEXT))    AS pos_of_a
        FROM character ch
        WHERE ch.name LIKE '%' || p_search || '%' -- % wildcard: match anywhere
            OR ch.name LIKE p_search || '_'      -- _ wildcard: one extra char after term
        ORDER BY ch.name;
END;
$$;

```

-- USAGE:

```

-- SELECT * FROM search_characters_by_name('man');
-- SELECT * FROM search_characters_by_name('Ultra');
-- SELECT * FROM search_characters_by_name('Iron');

```

```

-- =====
-- PROCEDURE 4 — get_story_arc_issues
--
-- REQUIREMENTS MET:
--   SELECT, INNER JOIN (x3), LEFT JOIN (x2), WHERE, LIKE '%',
--   COALESCE, AS, ORDER BY
--
-- EXTRA FEATURES:
--   LOWER() — used for case-insensitive arc title matching
--
-- Returns every issue in a story arc with creator credits.
-- COALESCE handles issues that have no creator on record.
-- Uses partial LIKE matching so you can pass part of a title.
-- =====

```

```

CREATE OR REPLACE FUNCTION get_story_arc_issues(p_arc_title VARCHAR)

```

```

RETURNS TABLE (
  arc_title  VARCHAR,
  part_num   SMALLINT,
  issue_title VARCHAR,
  publish_date DATE,
  series     VARCHAR,
  creator    TEXT,
  role       TEXT
)
LANGUAGE plpgsql AS $$
BEGIN
  RETURN QUERY
  SELECT
    sa.title                AS arc_title,
    isa.part_num,
    i.title                 AS issue_title,
    i.publish_date,
    s.title                 AS series,
    COALESCE(cr.first_name || ' ' || cr.last_name, 'Unknown') AS creator,
    COALESCE(ic.role::TEXT, 'N/A')                          AS role
  FROM issue_story_arc isa
  INNER JOIN story_arc   sa ON isa.arc_id   = sa.arc_id
  INNER JOIN issue       i ON isa.issue_id  = i.issue_id
  INNER JOIN series      s ON i.series_id   = s.series_id
  LEFT JOIN issue_creator ic ON i.issue_id  = ic.issue_id
  LEFT JOIN creator      cr ON ic.creator_id = cr.creator_id
  WHERE LOWER(sa.title) LIKE LOWER('%' || p_arc_title || '%')
  ORDER BY isa.part_num, ic.role;
END;
$$;

-- USAGE (pass a partial title — LIKE '%...%' will match it):
-- SELECT * FROM get_story_arc_issues('Dark');
-- SELECT * FROM get_story_arc_issues('Fall');
-- SELECT * FROM get_story_arc_issues('War');

-- =====
-- PROCEDURE 5 — summarize_publisher_stats
--
-- REQUIREMENTS MET:
-- SELECT, LEFT JOIN (x2), GROUP BY, ROLLUP, COUNT(), SUM(),
-- AVG(), MIN(), MAX(), ORDER BY DESC, COALESCE, AS
--

```

```
-- EXTRA FEATURES:
-- DISTINCT — used inside COUNT(DISTINCT ...) to avoid duplicates
-- ROUND() — rounds avg cover price to 2 decimal places
--
-- Returns a statistical summary per publisher, plus a grand
-- total row produced by ROLLUP. COALESCE labels that row.
-- =====
```

```
CREATE OR REPLACE FUNCTION summarize_publisher_stats()
RETURNS TABLE (
    publisher TEXT,
    total_series BIGINT,
    total_issues BIGINT,
    total_revenue NUMERIC,
    avg_cover_price NUMERIC,
    cheapest_issue NUMERIC,
    priciest_issue NUMERIC,
    earliest_issue DATE,
    latest_issue DATE
)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
        SELECT
            COALESCE(p.name::TEXT, '*** GRAND TOTAL ***') AS publisher,
            COUNT(DISTINCT s.series_id) AS total_series,
            COUNT(DISTINCT i.issue_id) AS total_issues,
            SUM(i.cover_price) AS total_revenue,
            ROUND(AVG(i.cover_price), 2) AS avg_cover_price,
            MIN(i.cover_price) AS cheapest_issue,
            MAX(i.cover_price) AS priciest_issue,
            MIN(i.publish_date) AS earliest_issue,
            MAX(i.publish_date) AS latest_issue
        FROM publisher p
        LEFT JOIN series s ON p.publisher_id = s.publisher_id
        LEFT JOIN issue i ON s.series_id = i.series_id
        GROUP BY ROLLUP(p.name)
        ORDER BY total_issues DESC;
END;
$$;

-- USAGE:
-- SELECT * FROM summarize_publisher_stats();
```

```

-- =====
-- PROCEDURE 6 — add_issue (PROCEDURE — modifies data)
--
-- REQUIREMENTS MET:
--  INSERT, SELECT, WHERE, AS
--
-- EXTRA FEATURES:
--  DECLARE      — declares the v_series_title local variable
--  RAISE EXCEPTION — thrown if series ID does not exist
--  RAISE NOTICE  — confirms successful insert to the console
--
-- Adds a new issue to an existing series.
-- Validates that the series exists before inserting.
-- =====

CREATE OR REPLACE PROCEDURE add_issue(
    p_series_id  INT,
    p_issue_number VARCHAR,
    p_title      VARCHAR,
    p_publish_date DATE,
    p_cover_price NUMERIC,
    p_page_count  SMALLINT,
    p_notes      TEXT DEFAULT NULL
)
LANGUAGE plpgsql AS $$
DECLARE
    v_series_title VARCHAR;
BEGIN
    SELECT title AS series_title
    INTO v_series_title
    FROM series
    WHERE series_id = p_series_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Series ID % does not exist.', p_series_id;
    END IF;

    INSERT INTO issue (series_id, issue_number, title, publish_date, cover_price, page_count,
notes)
    VALUES (p_series_id, p_issue_number, p_title, p_publish_date, p_cover_price,
p_page_count, p_notes);

    RAISE NOTICE 'Issue "%" (#%) added to series "%".', p_title, p_issue_number, v_series_title;

```



```

END;
$$;

-- USAGE:
-- CALL add_issue(1, '301', 'The New Age Begins', '2024-06-01', 4.99, 32, 'Start of a new
chapter.');
```

```

-- CALL add_issue(999, '1', 'Bad Insert', '2024-01-01', 3.99, 24); -- raises error

-- =====
-- PROCEDURE 7 — update_series_end_year (PROCEDURE — modifies data)
--
-- REQUIREMENTS MET:
-- UPDATE, SELECT, WHERE, AND, AS
--
-- EXTRA FEATURES:
-- DECLARE      — declares v_title, v_start_year, v_end_year variables
-- RAISE EXCEPTION — thrown for bad series ID or invalid year
-- RAISE NOTICE  — confirms the update to the console
-- LOWER()       — not used here but validation logic uses AND
--
-- Marks an ongoing series as ended. Validates that the
-- end year is strictly after the series start year.
-- =====

CREATE OR REPLACE PROCEDURE update_series_end_year(
  p_series_id INT,
  p_end_year INT
)
LANGUAGE plpgsql AS $$
DECLARE
  v_title  VARCHAR;
  v_start_year SMALLINT;
  v_end_year SMALLINT;
BEGIN
  SELECT title    AS series_title,
         start_year AS series_start,
         end_year  AS series_end
  INTO v_title, v_start_year, v_end_year
  FROM series
  WHERE series_id = p_series_id;

  IF NOT FOUND THEN
    RAISE EXCEPTION 'Series ID % does not exist.', p_series_id;
```

```

END IF;

IF p_end_year <= v_start_year THEN
    RAISE EXCEPTION 'End year (%) must be after start year (%)', p_end_year, v_start_year;
END IF;

IF v_end_year IS NOT NULL THEN
    RAISE NOTICE 'Series "%" already has end year %. Updating to %.', v_title, v_end_year,
p_end_year;
END IF;

UPDATE series
SET end_year = p_end_year
WHERE series_id = p_series_id;

RAISE NOTICE 'Series "%" end year set to %.', v_title, p_end_year;
END;
$$;

-- USAGE:
-- CALL update_series_end_year(1, 2024);
-- CALL update_series_end_year(999, 2024); -- raises error (bad ID)
-- CALL update_series_end_year(1, 1930); -- raises error (bad year)

-- =====
-- PROCEDURE 8 — delete_creator_credits (PROCEDURE — modifies data)
--
-- REQUIREMENTS MET:
-- DELETE, SELECT, WHERE, CONCAT, AS
--
-- EXTRA FEATURES:
-- DECLARE — declares v_name and v_deleted_count variables
-- GET DIAGNOSTICS — captures the number of deleted rows
-- RAISE EXCEPTION — thrown if creator ID does not exist
-- RAISE NOTICE — reports how many rows were removed
-- LOWER() — used inside CONCAT for name formatting
--
-- Removes all issue_creator rows for a given creator ID.
-- Reports how many rows were deleted via GET DIAGNOSTICS.
-- =====

CREATE OR REPLACE PROCEDURE delete_creator_credits(p_creator_id INT)
LANGUAGE plpgsql AS $$

```

```

DECLARE
    v_name      TEXT;
    v_deleted_count INT;
BEGIN
    SELECT CONCAT(first_name, ' ', last_name) AS full_name
    INTO v_name
    FROM creator
    WHERE creator_id = p_creator_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Creator ID % does not exist.', p_creator_id;
    END IF;

    DELETE FROM issue_creator
    WHERE creator_id = p_creator_id;

    GET DIAGNOSTICS v_deleted_count = ROW_COUNT;

    RAISE NOTICE 'Removed % credit(s) for creator "%" (ID %).', v_deleted_count, v_name,
    p_creator_id;
END;
$$;

-- USAGE:
-- CALL delete_creator_credits(5);
-- CALL delete_creator_credits(999); -- raises error

-- =====
-- PROCEDURE 9 — get_issues_with_character_count
--
-- REQUIREMENTS MET:
-- SELECT, INNER JOIN (x2), GROUP BY, HAVING, COUNT(),
-- SUM(), ORDER BY DESC, AS
--
-- EXTRA FEATURES:
-- (none — all features used are from the required list)
--
-- Returns issues featuring at least N characters.
-- Also sums total page count across matching issues.
-- Useful for identifying large crossover or event issues.
-- =====

```

```

CREATE OR REPLACE FUNCTION get_issues_with_character_count(p_min_characters INT
DEFAULT 3)
RETURNS TABLE (
    issue_id      INT,
    issue_title   VARCHAR,
    series        VARCHAR,
    publish_date  DATE,
    character_count BIGINT,
    total_pages   BIGINT
)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
        SELECT
            i.issue_id,
            i.title      AS issue_title,
            s.title      AS series,
            i.publish_date,
            COUNT(ic.character_id) AS character_count,
            SUM(i.page_count)    AS total_pages
        FROM issue i
        INNER JOIN series      s ON i.series_id = s.series_id
        INNER JOIN issue_character ic ON i.issue_id = ic.issue_id
        GROUP BY i.issue_id, i.title, s.title, i.publish_date
        HAVING COUNT(ic.character_id) >= p_min_characters
        ORDER BY character_count DESC;
END;
$$;

```

-- USAGE:

```

-- SELECT * FROM get_issues_with_character_count(3);
-- SELECT * FROM get_issues_with_character_count(5);

```

```

-- =====
-- PROCEDURE 10 — transfer_series_to_publisher (PROCEDURE — modifies data)
--
-- REQUIREMENTS MET:
--  UPDATE, SELECT, WHERE, COUNT(), AS
--
-- EXTRA FEATURES:
--  DECLARE      — declares v_from_name, v_to_name, v_series_count
--  RAISE EXCEPTION — thrown for bad publisher IDs or self-transfer
--  RAISE NOTICE  — confirms how many series were transferred

```

```

--
-- Bulk-moves all series from one publisher to another,
-- simulating a real-world licence transfer or acquisition.
-- Validates both publisher IDs and guards against self-transfer.
-- =====

CREATE OR REPLACE PROCEDURE transfer_series_to_publisher(
    p_from_publisher_id INT,
    p_to_publisher_id INT
)
LANGUAGE plpgsql AS $$
DECLARE
    v_from_name VARCHAR;
    v_to_name VARCHAR;
    v_series_count INT;
BEGIN
    SELECT name AS publisher_name
    INTO v_from_name
    FROM publisher
    WHERE publisher_id = p_from_publisher_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Source publisher ID % does not exist.', p_from_publisher_id;
    END IF;

    SELECT name AS publisher_name
    INTO v_to_name
    FROM publisher
    WHERE publisher_id = p_to_publisher_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Destination publisher ID % does not exist.', p_to_publisher_id;
    END IF;

    IF p_from_publisher_id = p_to_publisher_id THEN
        RAISE EXCEPTION 'Source and destination publisher cannot be the same.';
    END IF;

    SELECT COUNT(*) AS series_count
    INTO v_series_count
    FROM series
    WHERE publisher_id = p_from_publisher_id;

    IF v_series_count = 0 THEN

```

```
    RAISE NOTICE 'Publisher "%" has no series to transfer.', v_from_name;
    RETURN;
END IF;
```

```
UPDATE series
SET publisher_id = p_to_publisher_id
WHERE publisher_id = p_from_publisher_id;
```

```
    RAISE NOTICE 'Transferred % series from "%" to "%".', v_series_count, v_from_name,
v_to_name;
END;
$$;
```

```
-- USAGE:
-- CALL transfer_series_to_publisher(3, 1); -- move Image series to Marvel
-- CALL transfer_series_to_publisher(999, 1); -- raises error (bad source ID)
-- CALL transfer_series_to_publisher(1, 1); -- raises error (same publisher)
```

```
-- =====
-- PROCEDURE 11 — get_creator_string_analysis
--
-- REQUIREMENTS MET:
-- SELECT, WHERE, LIKE '_' wildcard, LIKE '%' wildcard, OR,
-- REPLACE, REVERSE, LENGTH, LENGTH(TRIM()), SUBSTR,
-- LEFT, RIGHT, POSITION, AS, ORDER BY
--
-- EXTRA FEATURES:
-- LOWER() — used inside POSITION() for case-insensitive search
--
-- Returns string-function breakdowns for creators whose
-- last name matches a wildcard pattern or contains the
-- search term anywhere within it.
-- =====
```

```
CREATE OR REPLACE FUNCTION get_creator_string_analysis(p_search VARCHAR)
RETURNS TABLE (
    full_name    TEXT,
    name_replaced TEXT,
    name_reversed TEXT,
    name_length  INT,
    trimmed_length INT,
    name_substr_1_5 TEXT,
    first_4_chars TEXT,
```

```

    last_4_chars TEXT,
    pos_of_e     INT
)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
    SELECT
        CONCAT(c.first_name, ' ', c.last_name) AS full_name,
        REPLACE(c.last_name, 'son', 'SON')    AS name_replaced,
        REVERSE(c.last_name)                  AS name_reversed,
        LENGTH(c.last_name)                   AS name_length,
        LENGTH(TRIM(c.last_name))              AS trimmed_length,
        SUBSTR(c.last_name, 1, 5)              AS name_substr_1_5,
        LEFT(c.last_name, 4)                   AS first_4_chars,
        RIGHT(c.last_name, 4)                  AS last_4_chars,
        POSITION('e' IN LOWER(c.last_name))     AS pos_of_e
    FROM creator c
    WHERE c.last_name LIKE p_search || ' _'    -- ' _' matches exactly one extra char
        OR c.last_name LIKE '%' || p_search || '%' -- '%' matches anywhere in name
    ORDER BY c.last_name;
END;
$$;

```

-- USAGE:

```

-- SELECT * FROM get_creator_string_analysis('Smith');
-- SELECT * FROM get_creator_string_analysis('John');

```

```

-- =====
-- PROCEDURE 12 — get_issue_rankings
--
-- REQUIREMENTS MET:
-- SELECT, INNER JOIN, WITH AS (CTE), OVER, ORDER BY DESC,
-- ROW_NUMBER(), RANK(), DENSE_RANK(), LAG(), LEAD(), AS
--
-- EXTRA FEATURES:
-- PARTITION BY — divides window function results per series
--                 so rankings reset for each series rather
--                 than running across the whole table
--
-- Uses a CTE to pre-aggregate the base data, then applies
-- window functions to rank issues by cover price within each
-- series, and shows the previous/next price via LAG/LEAD.
-- =====

```

```

CREATE OR REPLACE FUNCTION get_issue_rankings()
RETURNS TABLE (
    series      VARCHAR,
    issue_title  VARCHAR,
    cover_price  NUMERIC,
    publish_date DATE,
    row_num      BIGINT,
    price_rank   BIGINT,
    dense_price_rank BIGINT,
    prev_price   NUMERIC,
    next_price   NUMERIC
)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
        WITH issue_base AS (
            SELECT
                s.title      AS series_title,
                i.title      AS issue_title,
                i.cover_price,
                i.publish_date
            FROM issue i
            INNER JOIN series s ON i.series_id = s.series_id
        )
        SELECT
            ib.series_title      AS series,
            ib.issue_title,
            ib.cover_price,
            ib.publish_date,
            ROW_NUMBER() OVER (PARTITION BY ib.series_title ORDER BY ib.cover_price
DESC) AS row_num,
            RANK() OVER (PARTITION BY ib.series_title ORDER BY ib.cover_price DESC)
AS price_rank,
            DENSE_RANK() OVER (PARTITION BY ib.series_title ORDER BY ib.cover_price
DESC) AS dense_price_rank,
            LAG(ib.cover_price) OVER (PARTITION BY ib.series_title ORDER BY ib.publish_date)
AS prev_price,
            LEAD(ib.cover_price) OVER (PARTITION BY ib.series_title ORDER BY ib.publish_date)
AS next_price
        FROM issue_base ib
        ORDER BY ib.series_title, ib.cover_price DESC;
END;
$$;

```



```
-- USAGE:
-- SELECT * FROM get_issue_rankings();
-- SELECT * FROM get_issue_rankings() WHERE series = 'The Amazing Warriors';
```

```
-- =====
-- PROCEDURE 13 — get_genre_cross_publisher
--
-- REQUIREMENTS MET:
-- SELECT, CROSS JOIN, LEFT JOIN, GROUP BY, HAVING,
-- COUNT(), AS, ORDER BY DESC
--
-- EXTRA FEATURES:
-- DISTINCT — used in the subquery to get unique genres
--           without duplicates across the series table
--
-- Uses a CROSS JOIN to produce every possible combination
-- of publisher and genre, then filters to combinations that
-- actually exist — useful for reporting which publishers
-- cover which genres.
-- =====
```

```
CREATE OR REPLACE FUNCTION get_genre_cross_publisher()
RETURNS TABLE (
    publisher VARCHAR,
    genre      VARCHAR,
    series_count BIGINT
)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
        SELECT
            p.name          AS publisher,
            genre_list.genre,
            COUNT(s.series_id) AS series_count
        FROM publisher p
        CROSS JOIN (
            SELECT DISTINCT s2.genre
            FROM series s2
            WHERE s2.genre IS NOT NULL
        ) AS genre_list
        LEFT JOIN series s
            ON s.publisher_id = p.publisher_id
```

```

        AND s.genre      = genre_list.genre
    GROUP BY p.name, genre_list.genre
    HAVING COUNT(s.series_id) > 0
    ORDER BY p.name, series_count DESC;
END;
$$;

-- USAGE:
-- SELECT * FROM get_genre_cross_publisher();
-- SELECT * FROM get_genre_cross_publisher() WHERE genre = 'Horror';

-- =====
-- VIEW 1 — v_publisher_series_summary
--
-- REQUIREMENTS MET:
-- VIEW, SELECT, LEFT JOIN (x2), GROUP BY, COUNT(), AS,
-- ORDER BY DESC
--
-- EXTRA FEATURES:
-- DISTINCT — used inside COUNT(DISTINCT ...) to avoid
--             counting duplicate series or issue records
--
-- A reusable view showing how many series and issues each
-- publisher has, without writing the full JOIN every time.
-- =====

CREATE OR REPLACE VIEW v_publisher_series_summary AS
    SELECT
        p.name           AS publisher,
        COUNT(DISTINCT s.series_id) AS total_series,
        COUNT(DISTINCT i.issue_id)  AS total_issues
    FROM publisher p
    LEFT JOIN series s ON p.publisher_id = s.publisher_id
    LEFT JOIN issue i ON s.series_id    = i.series_id
    GROUP BY p.name
    ORDER BY total_issues DESC;

-- USAGE:
-- SELECT * FROM v_publisher_series_summary;
-- SELECT * FROM v_publisher_series_summary WHERE total_issues > 100;

-- =====

```

```

-- VIEW 2 — v_character_appearance_summary
--
-- REQUIREMENTS MET:
-- VIEW, SELECT, RIGHT JOIN, LEFT JOIN, COALESCE,
-- COUNT(), GROUP BY, ORDER BY DESC, AS
--
-- EXTRA FEATURES:
-- (none — all features used are from the required list)
--
-- Shows every character alongside their total appearances.
-- RIGHT JOIN ensures characters with zero appearances still
-- appear in the results with a count of 0.
-- =====

CREATE OR REPLACE VIEW v_character_appearance_summary AS
SELECT
    COALESCE(ch.name, 'Unknown') AS character_name,
    ch.alignment,
    p.name AS publisher,
    COUNT(ic.issue_id) AS total_appearances
FROM issue_character ic
RIGHT JOIN character ch ON ic.character_id = ch.character_id
LEFT JOIN publisher p ON ch.publisher_id = p.publisher_id
GROUP BY ch.name, ch.alignment, p.name
ORDER BY total_appearances DESC;

-- USAGE:
-- SELECT * FROM v_character_appearance_summary;
-- SELECT * FROM v_character_appearance_summary WHERE alignment = 'Villain';

```